COMP3311 Week 8 Wednesday Lecture

- Week 08 Wednesday
- Assignment 2
- Relational Algebra
- Exercise: Result Size and Scheme (i)
- Product
- Product (cont)
- Theta Join
- Exercise: Result Size and Scheme (ii)
- Examples of RelAlg Expressions
- Exercise: Mapping SQL to RelAlg
- Division
- Division (cont)
- Division (cont)
- Division (cont)
- DBMS Architecture
- DBMS Architecture (cont)
- DBMS Architecture (cont)

◆ Week 08 Wednesday

In today's lecture ...

- Relational Algebra, Query Execution

Things to do ...

- Quiz 5 due by 23:59 Friday 3 November
- Assignment 2 due by 23:59 Wednesday 15 November
- Must understand Python/Pyscopg2 by end of week

Things to note ...

- Assignment 2 spec and database now available

◆ Assignment 2

Things to note:

- last minute work ... **vwds2** overloaded ... jas/tutors overloaded
- send code in email as attachment, not as a screenshot
- make sure **helpers** + load without error
- don't **create view** within Python code
- write queries to answer questions like "Are there any ...?"

◆ Relational Algebra

Relational algebra: sets of tuples, operations mapping sets to sets

Core relational algebra operations:

- selection: choosing a subset of tuples/rows
- projection: choosing a subset of attributes/columns
- product, join: combining relations
- union, intersection, difference: combining relations
- rename: change names of relations/attributes

Common extensions include:

- aggregation, projection++, division, sort

◆ Relational Algebra (cont)

Have already looked at:

- rename, projection, selection: operations on a single relation
- union, intersection, difference: operations on two compatible relations

More interesting are operations:

- product, join: operations on two "non-compatible" relations

SQL implements all of the above, but only works on single relations

```
select a,b,c        projection
from r
where a > 5         selection
```


◆ Exercise: Result Size and Scheme (i)

Given the following:

- union-compatible tables R(A,B,C) (1000 tuples) and S(D,E,F) (500 tuples)
- Operation Usage Size Schema
  - rename $\rho_{T}(r)$ $|r|$ T
  - projection $\pi_{A,D}(r)$ $r/4$ $R'(X)$
  - selection $\sigma_{(A=B)}(r)$ selectivity R
- give result sizes and schemas for the following:
  - $Renames(T(A,B))$ $Proj(R,B)$ $Proj(R,C)$
  - $Sel(R=5)$ $Sel(A=5)$ $Sel(A=B)$ $Sel(B=5)$

◆ Product

Product combines information from two relations pairwise on tuples.

$r \times s = \{ (t_1 \mid t_1 \in r \wedge t_2 \in s)$, where $r(R)$ $s(S)$

If $t_1 = (A_1, ..., A_n)$ and $t_2 = (B_1, ..., B_m)$ then $(t_1 \mid t_2) = (A_1, ..., A_n, B_1, ..., B_m)$

Note: relations do not have to be union-compatible.

Result size is large: $|r \times s| = |r| \cdot |s|$ Schema: $R \cdot S$

Algorithmic view:

```
result = {}
for each tuple t1 in relation r
    for each tuple t2 in relation s
        result = result U (t1|t2)
```


◆ Product (cont)

Example of product:

R

A	B	C
a	m	x
b	n	y
c	p	z

S

X	Y
1	1
2	2

R x S

A	B	C	X	Y
a	m	x	1	1
a	m	x	2	2
b	n	y	1	1
b	n	y	2	2
c	p	z	1	1
c	p	z	2	2

◆ Natural Join

Natural join is a specialised product:

- containing only pairs that match on common attributes
- with one of each pair of common attributes eliminated

Consider relation schemas R(ABC,JK,LM), S(KL,MN,XYZ)

The natural join of relations r(R) and s(S) is defined as:

$r * s = \{ join(t_1, t_2) \mid t_1 \in r, t_2 \in s, match(t_1, t_2) \}$

where $match(t_1, t_2) = t_1[K] = t_2[K] \wedge t_1[L] = t_2[L] \wedge t_1[M] = t_2[M]$

Algorithmic view:

```
result = {}
for each tuple t1 in relation r
    for each tuple t2 in relation s
        if (matches(t1,t2))
            result = result U (combine(t1,t2))
```


◆ Natural Join (cont)

Example of natural join:

R

A	B	C
a	m	x
b	n	y
c	p	z

S

C	D
x	1
y	2
z	3

R Join S

A	B	C	D
a	m	x	1
b	n	y	2
c	p	z	3

◆ Theta Join

The theta join is a specialised product containing only pairs that match on a supplied condition C.

$r * s = \{ (t_1 \mid t_2) \mid t_1 \in r, t_2 \in s \wedge C(t_1 \mid t_2) \}$

where $r(R)$ $s(S)$

Examples: θ *1.join(R=2)* $r(2,...)$ θ *2.join(B<D=C+G)* $r(2)$

All attribute names are required to be distinct (or *natural join*)

Can be defined in terms of other RA operations:

$r *_C s = Join(C) \circ Sel(C) \circ r \times s$

Note that $r *_{true} s = r \times s$.

◆ Theta Join (cont)

Example theta join:

R

A	B	C
a	m	x
b	n	y
c	p	z

S

D	E
x	1
y	2
z	3

R *Join(C=D)* S

A	B	C	D	E
a	m	x	x	1
b	n	y	y	2
c	p	z	z	3

(Theta join is the most frequently-used join in SQL queries)

◆ Exercise: Result Size and Scheme (ii)

Given the following:

- union-compatible tables R(A,B,C) (1000 tuples) and S(C,D,E) (500 tuples)
- Operation Usage MinSize MaxSize Schema
  - product $r \times s$ $|r|.|s|$ $|r|.|s|$ $R \cdot S$
  - natural join $r * s$ 0 $|r|$ $R \cup S$
  - theta join $r *_{B<D} s$ 0 $|r|.|s|$ $R \cdot S$
- give result sizes and schemas for the following:
  - $r \times s$, $Join(s)$, $Join(R=C)(r)$, $Join(B=D)(r)$, $Join(B=D)(r)$

◆ Examples of RelAlg Expressions

Querying with relational algebra (join) ...

- Who drinks in Newtown bars?

```
Newtowndbars(dbar) = Del(addr=Newtown)(Bars)
Tmp = Frequenta Join(barname) Newtowndbars
Result(drinker) = Proj(drinker)(Tmp)
```

- Who drinks beers made by Carlton?

```
Carltonbeers = Del(brand=Carlton)(Beers)
Tmp = Likes Join(beer=name) Carltonbeers
Result(drinker) = Proj(drinker)(Tmp)
```

Reminder: projection removes duplicates

◆ Exercise: Mapping SQL to RelAlg

Give sequences of relational algebra operations to solve each of these:

- Bars where either Gernot or John drink.
- Bars where John drinks but Gernot doesn't
- Find bars that serve Newtown at the same price as the Coopers Bay Hotel charges for VB
- What beers are sold at the same price as CBH/Newt
- Which bar is most popular? (Most drinkers)
- Price of cheapest beer at each bar?
- Which beers are sold at all bars?

◆ Division

Consider two relation schemas R and S where $S \subseteq R$.

The division operation is defined on instances $r(R)$, $s(S)$ as:

$r/s = \{ t \mid t \in r(R-S) \wedge satisfy \}$

where $satisfy = \forall t_2 \in s(t_1 \cdot t \cdot t_2 \in r)$ $t_1 \in r(R-S)$

Operationally:

- consider each subset of tuples in r that match on t of $s(S)$
- for this subset of tuples, take the $(R-S)$ values from each
- if this covers all tuples in s, then include t in r/s the result

◆ Division (cont)

Example of division:

R

A	B
a	x
a	y
a	z
b	x
b	y
b	z

S

A	B
a	x
a	y
a	z

T

A
a
b

R / T

B
x
y

S / T

A
a

◆ Division (cont)

Querying with relational algebra (division) ...

Division handles queries that include the notion "for all".

E.g. Which beers are sold in all bars?

We can answer this as follows:

- generate a relation of beers and bars where they are sold
  - *r1 = Proj(beer,bar)(Sold)*
- generate a relation of all bars
  - *r2 = Rename2.1(bar)(Proj(name)(Bars))*
- find which beers appear in tuples with every bar
  - *res = r1 Div r2*

◆ DBMS Architecture

COMP3311 is not a course on DBMS Architecture (that's COMP3315)

But knowing just a little about how DBMSs work can help

- to avoid/fix inefficiencies in database applications
- ensure that there are no concurrency issues

DBMSs attempt to handle these issues in ...

- query processing (QP) ... methods for evaluating queries
- transaction processing (TxP) ... controlling concurrency

As a programmer, you give a list of control to the DBMS, but can

- use QP knowledge to make DB applications efficient
- use TxP knowledge to make DB applications safe

◆ DBMS Architecture (cont)

Our view of the DBMS so far ...

A machine to process SQL queries.

◆ DBMS Architecture (cont)

One view of DB engine: "relational algebra virtual machine"

selection σ_{C}() projection π_{T}() join (r, s)

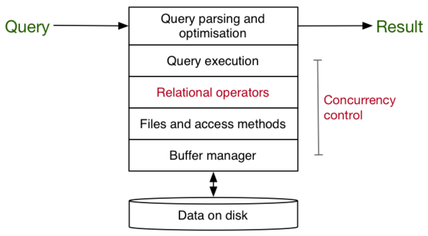
union (∪)	intersection (∩)	difference (-)
sort	insert	delete

For each of these operations:

- various data structures and algorithms are available
- DBMSs may provide only one, or may provide a choice

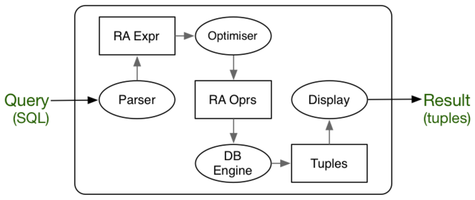
❖ DBMS Architecture (cont)

Layers in a DB Engine (Ramakrishnan's View)



❖ Query Evaluation

The path of a query through its evaluation:



❖ Mapping SQL to RA

Mapping SQL to relational algebra, e.g.

```
-- schema: R(a,b) S(c,d)
select a as x
from   R join S on (b=c)
where  d = 100
-- could be mapped to
Tmp1(a,b,c,d) = R Join[b=c] S
Tmp2(a,b,c,d) = Sel[d=100](Tmp1)
Tmp3(a)       = Proj[a](Tmp2)
Res(x)        = Rename[Res(x)](Tmp3)
```

In general:

- **`SELECT`** clause becomes *projection*
- **`WHERE`** condition becomes *selection* or *join*
- **`FROM`** clause becomes *join*

❖ Exercise: A Better SQL to RA Mapping

On the previous slide, we translated an SQL query as follows:

```
-- schema: R(a,b) S(c,d)
select a as x
from   R join S on (b=c)
where  d = 100
-- could be mapped to
Tmp1(a,b,c,d) = R Join[b=c] S
Tmp2(a,b,c,d) = Sel[d=100](Tmp1)
Tmp3(a)       = Proj[a](Tmp2)
Res(x)        = Rename[Res(x)](Tmp3)
```

Suggest a more efficient approach (based on likely size of intermediate results)

❖ Mapping Example

The query: *Courses with more than 100 students in them?*

Can be expressed in SQL as

```
select   s.id, s.code
from     Course c, Subject s, Enrolment e
where    c.id = e.course and c.subject = s.id
group by s.id, s.code
having   count(*) > 100;
```

and might be compiled to

```
Result =
Project[id,code](
   GroupSelect[size>100] (
      GroupBy[id,code] (
         Subject Join[s.id=c.subject]
         (Course Join[c.id=e.course] Enrolment)
)  )  )
```


Produced: 1 Nov 2023